Dual-Channel Heartbeat / AC / CCL / TC Design

ATSAMC21J18A-AU operational design target.
Date: 2026-05-14

This document is meant to be translated directly into firmware procedures and schematic/netlist checks. It only uses facts checked against the local CMSIS headers, existing scope experiments, and the `llm-wiki` notes where they match the headers.

1. Required Behavior

Common hardware:

- A 24 MHz crystal drives the heartbeat time base.
- TCC0 generates three phase-aligned PWM signals at 375 kHz:
  - `HB_1_8` : duty 1/8
  - `HB_7_8` : duty 7/8
  - `HB_1_2` : duty 1/2
- A common window counter counts heartbeat periods and raises:
  - a compare interrupt before the end of the window
  - an overflow interrupt and overflow event at the window boundary

Per channel:

- One analog input is observed by one AC comparator and by the ADC.
- The AC comparator output is routed to an external D flip-flop.
- `HB_1_2` clocks both D flip-flops.
- The DFF output is the synchronized comparator state:

```
AC_SYNC = DFF.Q sampled on the active edge of HB_1_2
```

- One CCL LUT selects the channel reference waveform:

```
REF_OUT = AC_SYNC ? HB_1_8 : HB_7_8
```

- A second CCL LUT generates the counted pulse stream:

```
COUNT_PULSE = AC_SYNC & HB_1_2
```

- Each `COUNT_PULSE` stream is counted by a 32-bit TC pair.

2. Hard Resource Facts

- TCC0 is needed for the three heartbeat PWM outputs.
- The device has four CCL LUTs. This design uses all four.
- The only confirmed 32-bit TC pairs in this project are:
  - `TC0+TC1`
  - `TC2+TC3`
- `TC4_MASTER` is 0 in the headers and there is no usable `TC5` instance in this project configuration. Therefore there is no third independent hardware TC32 pair available for the common window while both channels are counted in hardware.
- The operational allocation is therefore:
  - `TC0+TC1` : Channel A 32-bit event counter
  - `TC2+TC3` : Channel B 32-bit event counter
  - `TCC1` : common heartbeat-window counter
- `TCC1` is 24-bit, not 32-bit. With event counting at 375 kHz the maximum direct hardware window is `2^24` heartbeat periods, about 44.7 s. If the common counter must be a true third hardware 32-bit counter, this MCU/resource allocation cannot provide it without sacrificing one channel counter.

3. Concrete Pin And Net Allocation

Common Heartbeat Nets

Net	MCU source	Pin	Mux	Notes
`HB_1_8`	`TCC0/WO0`	PA08	E	Optional physical output/debug. Also used internally by LUT0/LUT3 through `INSEL=TCC`.

Net	MCU source	Pin	Mux	Notes
`HB_7_8`	`TCC0/WO1`	PA09	E	Optional physical output/debug. Also used internally by LUT0/LUT3 through `INSEL=TCC`.
`HB_1_2`	`TCC0/WO2`	PA10	F	Physical clock net for both external DFFs and for CCL loopback inputs.

Important constraint: PA10 cannot be both `TCC0/WO2` output and `CCL_IN5` input at the same time. If `HB_1_2` must enter LUT1/LUT2, route PA10 externally to separate CCL input pads.

Required external fanout for `HB_1_2`:

Source net	Destination
PA10 `TCC0/WO2`	DFF_A.CLK
PA10 `TCC0/WO2`	DFF_B.CLK
PA10 `TCC0/WO2`	PB10 `CCL_IN5`, used by LUT1 IN2
PA10 `TCC0/WO2`	PA24 `CCL_IN8`, used by LUT2 IN2

Channel A

Signal	Pin/net	Mux	Peripheral role
Analog input A	PA05	B	`AC_AIN1`, `ADC0_AIN5`
Comparator output A	PA12	H	`AC_CMP0`, routed to DFF_A.D
DFF_A.Q to mux LUT	PA18	I	`CCL_IN2`, LUT0 IN2
DFF_A.Q to gate LUT	PA30	I	`CCL_IN3`, LUT1 IN0
Reference output A	PA07	I	`CCL_OUT0`, LUT0 output
Count/debug output A	PA11	I	`CCL_OUT1`, LUT1 output, optional physical debug

The DFF output must fan out to two MCU pins because LUT0 and LUT1 are separate CCL cells. There is no internal "reuse this IO input in another LUT" path.

Channel B

Signal	Pin/net	Mux	Peripheral role
Analog input B	PA06	B	`AC_AIN2` , `ADC0_AIN6`
Comparator output B	PA13	H	`AC_CMP1` , routed to DFF_B.D
DFF_B.Q to mux LUT	PB16	I	`CCL_IN11` , LUT3 IN2
DFF_B.Q to gate LUT	PA22	I	`CCL_IN6` , LUT2 IN0
Reference output B	PB17	I	`CCL_OUT3` , LUT3 output
Count/debug output B	PA25	I	`CCL_OUT2` , LUT2 output, optional physical debug

The two analog pins are both on ADC0. That is fine for sequential ADC sampling. If simultaneous ADC sampling is required, the pin allocation must change to use ADC0 plus ADC1.

4. Peripheral Allocation

Block	Allocation
XOSC	24 MHz crystal on PA14/PA15
GCLK0	48 MHz OSC48M, CPU and CCL
GCLK1	24 MHz XOSC, TCC0/TCC1
GCLK2	24 MHz / 64 = 375 kHz, optional AC clock
TCC0	Heartbeat PWM generator
TCC1	Common window counter, event-counting TCC0 overflow events
AC COMP0	Channel A comparator, PA05 positive input
AC COMP1	Channel B comparator, PA06 positive input
CCL LUT0	Channel A reference mux
CCL LUT1	Channel A gated count pulse
CCL LUT2	Channel B gated count pulse

Block	Allocation
CCL LUT3	Channel B reference mux
TC0+TC1	Channel A 32-bit event counter
TC2+TC3	Channel B 32-bit event counter
EVSYS CH0	TCC0 overflow to TCC1 event input 0
EVSYS CH1	CCL LUTOUT1 to TC0 event input
EVSYS CH2	CCL LUTOUT2 to TC2 event input

5. TCC0 Heartbeat Procedure

Enable clocks:

```
MCLK->APBCMASK.reg |= MCLK_APBCMASK_TCC0;
GCLK->PCHCTRL[TCC0_GCLK_ID].reg = GCLK_PCHCTRL_GEN_GCLK1 | GCLK_PCHCTRL_CHEN;
```

Configure TCC0:

Register	Value	Result
`CTRLA.PRESCALER`	`DIV1`	TCC0 runs at 24 MHz
`WAVE.WAVEGEN`	`NPWM`	Single-slope PWM
`PER`	63	64 ticks per heartbeat period
`CC[0]`	8	`HB_1_8` , high for 8/64 ticks
`CC[1]`	56	`HB_7_8` , high for 56/64 ticks
`CC[2]`	32	`HB_1_2` , high for 32/64 ticks
`EVCTRL.OVFEO`	1	TCC0 overflow event marks one heartbeat period

Expected waveform:

```
f_heartbeat = 24 MHz / 64 = 375 kHz
T_heartbeat = 2.666666 us
HB_1_8 high time = 333.333 ns
HB_7_8 high time = 2.333333 us
HB_1_2 high time = 1.333333 us
```

Configure pins:

```
PA08 mux E = TCC0/WO0
PA09 mux E = TCC0/WO1
PA10 mux F = TCC0/WO2
```

6. TCC1 Window Counter Procedure

Use TCC1 as a heartbeat-period event counter. It counts TCC0 overflow events, not 24 MHz ticks.

Enable clocks:

```
MCLK->APBCMASK.reg |= MCLK_APBCMASK_TCC1;
// TCC0_GCLK_ID and TCC1_GCLK_ID are both 28 on SAMC21J18A.
// The GCLK1 assignment made for TCC0 also clocks TCC1.
```

EVSYS route:

EVSYS channel	Generator	User	Path
0	`EVSYS_ID_GEN_TCC0_OVF` = 35	`EVSYS_ID_USER_TCC1_EV_0` = 15	`ASYNC`

```
EVSYS->CHANNEL[0].reg =
    EVSYS_CHANNEL_EVGEN(EVSYS_ID_GEN_TCC0_OVF) |
    EVSYS_CHANNEL_PATH_ASYNCHRONOUS;
EVSYS->USER[EVSYS_ID_USER_TCC1_EV_0].reg = EVSYS_USER_CHANNEL(1); // channel 0 + 1
```

TCC1 setup for a window of `N` heartbeat periods:

Register	Value	Notes
`CTRLA.PRESCALER`	`DIV1`	Clock still required by the TCC peripheral

Register	Value	Notes
`WAVE.WAVEGEN`	`NFRQ`	No waveform output needed
`PER`	`N - 1`	Direct range: `3 <= N <= 0x1000000` for the prep interrupt below
`CC[0]`	`N - 2`	Prep interrupt at the start of the penultimate interval
`EVCTRL`	`TCEI0	EVACT0_COUNTEV
`INTENSET`	`MC0	OVF`

Operational definition:

```
TCC0 OVF marks the boundary between heartbeat intervals.
The active intervals in a window are numbered 0 .. N-1.
MC0 at CC=N-2 means: interval N-2, the penultimate interval, has started.
OVF means: interval N-1 has ended and the N-period window is closed.
```

The `CC[0]=N-2` choice makes the compare interrupt a deterministic preparation point before the final interval. If the firmware later needs "start of last interval" rather than "start of penultimate interval", validate `CC[0]=N-1` on the scope because that places compare at the top count and may be implementation-timing sensitive.

TCC1 overflow event is available as generator `EVSYS_ID_GEN_TCC1_OVF` = 42 if a downstream peripheral needs a hardware window-boundary event.

7. AC And ADC Procedure

Enable:

```
MCLK->APBCMASK.reg |= MCLK_APBCMASK_AC;
GCLK->PCHCTRL[AC_GCLK_ID].reg = GCLK_PCHCTRL_GEN_GCLK2 | GCLK_PCHCTRL_CHEN;
```

Channel A comparator:

Field	Value
Comparator	`COMP0`

Field	Value
Positive input	`MUXPOS_PIN1` , PA05 / `AC_AIN1`
Negative input	Project threshold: `VSCALE` , `INTREF` , or DAC/reference path
Speed	`SPEED_HIGH`
Filter	`FLEN_OFF`
Output	`OUT_ASYNC` recommended when the external DFF does the synchronization
Pad output	PA12 mux H, `AC_CMP0` , to DFF_A.D
ADC	PA05 / `ADC0_AIN5`

Channel B comparator:

Field	Value
Comparator	`COMP1`
Positive input	`MUXPOS_PIN2` , PA06 / `AC_AIN2`
Negative input	Project threshold: `VSCALE` , `INTREF` , or DAC/reference path
Speed	`SPEED_HIGH`
Filter	`FLEN_OFF`
Output	`OUT_ASYNC` recommended when the external DFF does the synchronization
Pad output	PA13 mux H, `AC_CMP1` , to DFF_B.D
ADC	PA06 / `ADC0_AIN6`

Do not label PA05/PA06 as `ADC0_AIN1` / `ADC0_AIN2` ; the headers define them as `ADC0_AIN5` and `ADC0_AIN6` .

8. External DFF Requirements

Use one dual D-type flip-flop package, for example half A and half B of a 74HC74-family part, or any voltage-compatible faster equivalent.

DFF pin	Channel A	Channel B
`D`	PA12 `AC_CMP0`	PA13 `AC_CMP1`
`CLK`	`HB_1_2` from PA10	`HB_1_2` from PA10
`Q`	Fanout to PA18 and PA30	Fanout to PB16 and PA22
`/PRE`	Pull inactive high	Pull inactive high
`/CLR`	Pull inactive high	Pull inactive high

Use the DFF clock edge consistently in firmware notes and scope procedures. The default assumption in this design is that `Q` updates on the rising edge of `HB_1_2`.

9. CCL Procedure

Enable:

```
MCLK->APBCMASK.reg |= MCLK_APBCMASK_CCL;
GCLK->PCHCTRL[CCL_GCLK_ID].reg = GCLK_PCHCTRL_GEN_GCLK0 | GCLK_PCHCTRL_CHEN;
```

Disable both CCL sequential pairs:

```
CCL->SEQCTRL[0].reg = 0;
CCL->SEQCTRL[1].reg = 0;
```

Truth-table bit index is:

```
bit_index = (IN2 << 2) | (IN1 << 1) | IN0
```

LUT0 - Channel A Reference Mux

Purpose:

```
REF_A = Q_A ? HB_1_8 : HB_7_8
```

LUT input	INSEL	Source
IN0	`TCC`	TCC0/WO0 = `HB_1_8`
IN1	`TCC`	TCC0/WO1 = `HB_7_8`
IN2	`IO`	PA18 / `CCL_IN2` = DFF_A.Q

`TRUTH = 0xAC`.

Output:

```
LUT0 -> CCL_OUT0 -> PA07 mux I = REF_A
```

LUT3 - Channel B Reference Mux

Local experiment notes confirm that LUT3 wraps to TCC0 for `INSEL=TCC`, so it can use TCC0/WO0 and TCC0/WO1 like LUT0.

Purpose:

```
REF_B = Q_B ? HB_1_8 : HB_7_8
```

LUT input	INSEL	Source
IN0	`TCC`	TCC0/WO0 = `HB_1_8`
IN1	`TCC`	TCC0/WO1 = `HB_7_8`
IN2	`IO`	PB16 / `CCL_IN11` = DFF_B.Q

`TRUTH = 0xAC`.

Output:

```
LUT3 -> CCL_OUT3 -> PB17 mux I = REF_B
```

If the desired polarity is instead `Q ? HB_7_8 : HB_1_8`, use `TRUTH = 0xCA`.

LUT1 - Channel A Count Pulse

Purpose:

```
COUNT_A = Q_A & HB_1_2
```

LUT input	`INSEL`	Source
IN0	`IO`	PA30 / `CCL_IN3` = DFF_A.Q
IN1	`MASK`	Constant 0
IN2	`IO`	PB10 / `CCL_IN5` = external loopback from PA10 `HB_1_2`

`TRUTH = 0x20` because IN1 is masked to 0. If IN1 is not masked, the equivalent full truth table for `IN2 & IN0` is `0xA0`.

Required bits:

```
LUTEO = 1
ENABLE = 1
```

Output/event:

```
LUT1 -> CCL_OUT1 -> PA11 mux I = optional COUNT_A debug pin
LUT1 event generator -> EVSYS_ID_GEN_CCL_LUTOUT_1 = 83
```

LUT2 - Channel B Count Pulse

Purpose:

```
COUNT_B = Q_B & HB_1_2
```

LUT input	`INSEL`	Source
IN0	`IO`	PA22 / `CCL_IN6` = DFF_B.Q
IN1	`MASK`	Constant 0
IN2	`IO`	PA24 / `CCL_IN8` = external loopback from PA10 `HB_1_2`

`TRUTH = 0x20` because IN1 is masked to 0.

Required bits:

```
LUTEO = 1
ENABLE = 1
```

Output/event:

```
LUT2 -> CCL_OUT2 -> PA25 mux I = optional COUNT_B debug pin
LUT2 event generator -> EVSYS_ID_GEN_CCL_LUTOUT_2 = 84
```

CCL Summary

LUT	Function	IN2	IN1	IN0	Truth
0	`REF_A` mux	PA18 `Q_A`	TCC0/WO1 `HB_7_8`	TCC0/WO0 `HB_1_8`	`0xAC`
1	`COUNT_A` gate	PB10 `HB_1_2`	MASK	PA30 `Q_A`	`0x20`
2	`COUNT_B` gate	PA24 `HB_1_2`	MASK	PA22 `Q_B`	`0x20`
3	`REF_B` mux	PB16 `Q_B`	TCC0/WO1 `HB_7_8`	TCC0/WO0 `HB_1_8`	`0xAC`

10. EVSYS And TC32 Counter Procedure

Enable:

```
MCLK->APBCMASK.reg |= MCLK_APBCMASK_EVSYS
                    | MCLK_APBCMASK_TC0
                    | MCLK_APBCMASK_TC1
                    | MCLK_APBCMASK_TC2
                    | MCLK_APBCMASK_TC3;
GCLK->PCHCTRL[TC0_GCLK_ID].reg = GCLK_PCHCTRL_GEN_GCLK0 | GCLK_PCHCTRL_CHEN;
GCLK->PCHCTRL[TC2_GCLK_ID].reg = GCLK_PCHCTRL_GEN_GCLK0 | GCLK_PCHCTRL_CHEN;
```

Event routes:

EVSYS channel	Generator	User	Path
1	`EVSYS_ID_GEN_CCL_LUTOUT_1` = 83	`EVSYS_ID_USER_TC0_EVU` = 23	`ASYNC`
2	`EVSYS_ID_GEN_CCL_LUTOUT_2` = 84	`EVSYS_ID_USER_TC2_EVU` = 25	`ASYNC`

```
EVSYS->CHANNEL[1].reg =
    EVSYS_CHANNEL_EVGEN(EVSYS_ID_GEN_CCL_LUTOUT_1) |
    EVSYS_CHANNEL_PATH_ASYNCHRONOUS;
EVSYS->USER[EVSYS_ID_USER_TC0_EVU].reg = EVSYS_USER_CHANNEL(2); // channel 1 + 1

EVSYS->CHANNEL[2].reg =
    EVSYS_CHANNEL_EVGEN(EVSYS_ID_GEN_CCL_LUTOUT_2) |
    EVSYS_CHANNEL_PATH_ASYNCHRONOUS;
EVSYS->USER[EVSYS_ID_USER_TC2_EVU].reg = EVSYS_USER_CHANNEL(3); // channel 2 + 1
```

Use asynchronous EVSYS path for `TC_EVACT_COUNT`. Existing local tests showed that TC event counting worked with `PATH_ASYNCHRONOUS`; resynchronized path did not work for that use.

Configure `TC0+TC1` and `TC2+TC3`:

Register	Value
`CTRLA.MODE`	`COUNT32` on the master (`TC0`, `TC2`)
`CTRLA.PRESCALER`	`DIV1`
`EVCTRL`	`TCEI
`COUNT`	Reset to 0 at window start

Count meaning:

```
TC0 COUNT32 = number of rising events from COUNT_A
TC2 COUNT32 = number of rising events from COUNT_B
```

At TCC1 overflow:

1. Clear `TCC1->INTFLAG.OVF` by writing 1.
2. Issue `READSYNC` on `TC0->COUNT32` and `TC2->COUNT32`.
3. Read `TC0->COUNT32.COUNT` and `TC2->COUNT32.COUNT`.
4. Reset both 32-bit counters to 0 for the next window.

Boundary note: the simple ISR read/reset method must be validated on the scope. The first count pulse of the next window is close to the TCC1 overflow boundary. If zero-ambiguity continuous windows are required, add a hardware capture/reset scheme or an explicit blanking interval; a single TC event input cannot both count CCL pulses and also consume the TCC1 overflow event as a reset trigger.

11. Firmware Initialization Order

1. Start OSC48M/GCLK0 as already done by `sys_init()` .
2. Start the 24 MHz XOSC with timeout and configure `GCLK1` .
3. Configure `GCLK2 = XOSC / 64` if AC uses that clock.
4. Enable APB masks for EVSYS, TCC0, TCC1, TC0, TC1, TC2, TC3, AC, CCL, ADC0.
5. Configure all pins before enabling their peripheral outputs.
6. Configure TCC0 heartbeat, including `OVFEO` .
7. Configure EVSYS channel 0 from TCC0 OVF to TCC1 EV0.
8. Configure TCC1 window counter, but leave it disabled until counters are reset.
9. Configure AC COMP0/COMP1 and verify `READY0/READY1` .
10. Configure CCL LUT0..LUT3 with `LUTEO` on LUT1/LUT2.
11. Configure EVSYS channels 1 and 2 from CCL LUTOUT1/2 to TC0/TC2.
12. Configure TC0+TC1 and TC2+TC3 as COUNT32 event counters and reset counts.
13. Enable CCL, TCs, TCC1, then TCC0 in a deterministic order for the first test.
14. Enable `TCC1_IRQn` ; in the handler service `MC0` and `OVF` separately.

12. Bring-Up Validation Checklist

Scope checks, in order:

1. PA08, PA09, PA10 show 375 kHz waveforms with 1/8, 7/8, 1/2 duty.
2. PA10 loopback appears on PB10 and PA24 as valid CCL input levels.
3. PA12 follows COMP0 and PA13 follows COMP1 with `OUT_ASYNC` .
4. DFF_A.Q only changes on PA10 clock edges.
5. DFF_B.Q only changes on PA10 clock edges.
6. PA07 produces `Q_A ? HB_1_8 : HB_7_8` .
7. PB17 produces `Q_B ? HB_1_8 : HB_7_8` .
8. PA11 produces `Q_A & HB_1_2` .
9. PA25 produces `Q_B & HB_1_2` .
10. For forced `Q_A=1` , TC0 count over a known window is approximately `N` .
11. For forced `Q_A=0` , TC0 count is 0.
12. For forced `Q_B=1` , TC2 count over a known window is approximately `N` .
13. For forced `Q_B=0` , TC2 count is 0.
14. TCC1 `MC0` fires before `OVF` .

15. TCC1 `OVF` fires once per configured window and the serial report shows stable TC0/TC2 readings over repeated windows.

13. Known Open Decisions

- Comparator negative input source and threshold are not chosen here. Pick `VSCALE`, `INTREF`, or an external/reference path per analog requirement.
- ADC scheduling is not defined here. PA05 and PA06 both use ADC0, so firmware must sample them sequentially unless pins are reassigned.
- DFF electrical family is not locked. Confirm VIH/VIL and propagation delay at board voltage.
- `CC[0]=N-2` for the TCC1 prep interrupt means "start of the penultimate interval" with zero-based interval numbering. If the intended semantic is "start of the last interval", validate `CC[0]=N-1` on hardware.
- TC2+TC3 COUNT32 is locally observed as functional, but the family errata notes TC2/TC3 pairing issues. Treat that as a board qualification item before freezing the production design.